

SRI KRISHNADEVARAYA EDUCATIONAL TRUST

SIR M. VISVESVARAYA INSTITUTE OF TECHNOLOGY

Krishnadevarayanagar, Hunasamaranahalli, International Air Port Road, Bangalore-562 157.

(Affiliated to Visvesvaraya Technological University, Recognised by AICTE & Accredited by National Board of Accreditation, New Delhi. An ISO 9001 : 2008 Certified Institution)

Ph. : 080-2846 7248, 2847 7024/25/26 Fax : 080-2846 7081

E-mail : principal@sirmvit.edu; sirmvitbgl@gmail.com, Web : www.sirmvit.edu



DEPARTMENT OF CSE (IOT & CYBER SECURITY INCLUDING BLOCK CHIAN TECHNOLOGY)

Project Management (with Git) LABORATORY

1BCSL307A– 3rd SEMESTER B.E

Academic Year – 2025-26

LAB MANUAL

Prepared by:
Pooja B
Assistant Professor
Dept. of IOT

Under the Guidance of:
DR. SAVITHA CHOUDARY
Prof. and Head
Dept. of IOT

Department Vision and MISSION

VISION

Building a Proficient Internet of Things Engineers with core values to solve **real time** problems.

MISSION

- M1: To offer state-of-the-art Internet of Things education imparting skills for building Cutting-edge and innovative applications with block chain and cyber security.
- M2: To inculcate ethically and socially committed Internet of Things professionals through Value-added courses.
- M3: Promoting research blend among students and enabling continuous learning

1. Course Description

Project Management (with Git) introduces students to the fundamentals of version control and Git-based project management. The course provides hands-on experience in creating and managing Git repositories, tracking changes, creating and merging branches, handling remote repositories, and collaborating in distributed development environments. Students also learn Git stashing, rebasing, cherry-picking, tagging, and techniques for analysing and modifying Git history. The course emphasizes practical Git skills required for effective source-code management and collaborative software development.

2. Course Objectives

The course aims to enable students to:

1. Understand the fundamentals of Git and distributed version control systems and their role in software project management.
2. Develop practical skills for creating, configuring, and managing Git repositories and tracking changes.
3. Apply branching, merging, and stashing techniques for managing parallel development activities.
4. Use remote repositories and collaboration techniques for distributed software development.
5. Perform advanced Git operations such as rebasing, cherry-picking, tagging, and reverting commits while maintaining repository integrity.

3. Course Outcomes (CO)

CO No.	Course Outcome
CO1	Demonstrate the use of basic Git repository commands for version control operations.
CO2	Implement branch management strategies for collaborative software development.
CO3	Apply collaboration techniques using remote repositories and distributed version control systems.
CO4	Configure Git tags, releases, and advanced Git operations for effective project management.
CO5	Modify and analyse Git history to maintain repository integrity and support project requirements.

PO	Program Outcomes (PO)	Level	Proficiency Assessed By
PO1	Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.	2	Practical Hands-on Sessions, Lab Exercises, Viva-Voce
PO2	Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using engineering principles.	2	Assignments, Practical Hands-on Sessions, Viva-Voce
PO3	Design/development of solutions: Design solutions for complex engineering problems and system components to meet specified needs.	3	Practical Hands-on Sessions, Demonstration, Mini Project
PO4	Conduct investigations of complex problems: Use research-based knowledge and methods to analyze, interpret, and validate engineering solutions.	3	Practical Hands-on Sessions, Lab Exercises, Demonstration
PO5	Modern tool usage: Create, select, and apply modern engineering and IT tools with an understanding of their limitations.	3	Practical Hands-on Sessions, Demonstration, Viva-Voce
PO6	The engineer and society: Apply contextual knowledge to assess societal, legal, safety, and professional responsibilities.	2	Demonstration, Viva-Voce
PO7	Environment and sustainability: Understand the impact of engineering solutions in societal and environmental contexts and the need for sustainable development.	2	Assignments, Case Study, Demonstration
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities in engineering practice.	2	Viva-Voce, Assignments, Demonstration
PO9	Individual and team work: Function effectively as an individual and as a member or leader in multidisciplinary teams.	2	Team-based Practical Hands-on Sessions, Demonstration
PO10	Communication: Communicate effectively through technical reports, documentation, presentations, and demonstrations.	2	Lab Record, PPT Presentation, Demonstration
PO11	Project management and finance: Apply engineering and management principles to manage projects in multidisciplinary environments.	2	Mini Project, Practical Hands-on Sessions, Demonstration
PO12	Life-long learning: Recognize the need for and engage in independent and life-long learning in the context of technological change.	2	Self-Learning Assignment, Viva-Voce, Practical Hands-on Sessions

Course Code	1BCSL307A	Scheme	2025
Teaching Hours/Week (L:T:P: S)	0:0:2	Semester	3
Type of Course	AEC	CIE Marks	50
Total Hours of L:T:P:SL:TW	0::0:28:2:30	SEE Marks	50
Credits	01	Total Marks	100
Type of Examination	Practical	Exam Hours	03

Course outcome (Course Skill Set)

At the end of the course, the student will be able to:

1. Demonstrate the use of basic Git repository commands for version control operations.
2. Implement branch management strategies for collaborative software development
3. Apply collaboration techniques using remote repositories and distributed version control systems.
4. Configure Git tags, releases, and advanced Git operations for effective project management.
5. Modify Git history to maintain repository integrity and support project requirements.

SLNO	Experiments
1	Setting Up and Basic Commands: Initialize a new Git repository in a directory. Create a new file and add it to the staging area, and commit the changes with an appropriate commit message
2	Creating and Managing Branches: Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature branch" into "master."
3	Creating and Managing Branches: Write the commands to stash your changes, switch branches, and then apply the stashed changes
4	Collaboration and Remote Repositories: Clone a remote Git repository to your local machine.
5	Collaboration and Remote Repositories: Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.
6	Collaboration and Remote Repositories: Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge.
7	Git Tags and Releases: Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository.
8	Advanced Git Operations: Write the command to cherry-pick a range of commits from "source-branch" to the current branch.
9	Analysing and Changing Git History: Given a commit ID, how would you use Git to view the details of that specific commit, including the author, date, and commit message?
10	Analysing and Changing Git History: Write the Git command to display all commits made by the author "JohnDoe" between "January 1, 2025" and "December 31, 2025."
11	Analysing and Changing Git History: Write the command to display the last five commits in the repository's history.
12	Analysing and Changing Git History: Write the command to undo the changes introduced by the commit with the ID "abc123".

GIT LABORATORY MANUAL

12 Git Programs

Program 1: Setting Up and Basic Git Commands

Aim: Initialize a new Git repository, create a file, add it to the staging area, and commit the changes with an appropriate commit message.

Procedure

Step 1: Create a directory

```
mkdir git-lab  
cd git-lab
```

Step 2: Initialize the repository

```
git init
```

Step 3: Create a file

```
echo "# Git Lab" > README.md
```

Step 4: Check repository status

```
git status
```

Step 5: Add the file to the staging area

```
git add README.md
```

Step 6: Check status

```
git status
```

Step 7: Commit the changes

```
git commit -m "Initial commit"
```

Step 8: View the commit history

```
git log
```

Result

The Git repository was initialized successfully, and the file was added to the staging area and committed.

Program 2: Creating and Managing Branches

Aim: Create a branch named "feature-branch", switch to it, switch back to "master", and merge the feature branch into master.

Procedure

Step 1: Check existing branches

```
git branch
```

Step 2: Create the feature branch

```
git branch feature-branch
```

Step 3: Switch to the feature branch

```
git switch feature-branch
```

Alternative command

```
git checkout feature-branch
```

Step 4: Create a feature file

```
echo "Feature work" > feature.txt
```

Step 5: Add and commit the feature

```
git add feature.txt
```

```
git commit -m "Add feature"
```

Step 6: Switch to master

```
git switch master
```

If the repository uses main

```
git switch main
```

Step 7: Merge the feature branch

```
git merge feature-branch
```

Step 8: View the history

```
git log --oneline --all
```

Result

The feature-branch was successfully merged into the master branch.

Program 3: Stashing and Applying Changes

Aim: Temporarily save uncommitted changes, switch branches, and then apply the stashed changes.

Procedure

Step 1: Switch to the feature branch

```
git switch feature-branch
```

Step 2: Modify a file

```
echo "New changes" >> feature.txt
```

Step 3: Check the changes

```
git status
```

Step 4: Stash the changes

```
git stash
```

Step 5: Switch to another branch

```
git switch master
```

Step 6: Return to the feature branch

```
git switch feature-branch
```

Step 7: Apply the stashed changes

```
git stash apply
```

Step 8: Check the status

```
git status
```

Notes

- View saved stashes: `git stash list`
- Apply and remove the latest stash: `git stash pop`
- Remove a stash after applying it: `git stash drop`

Result

The uncommitted changes were successfully stashed and later restored.

Program 4: Cloning a Remote Git Repository

Aim: Clone a remote Git repository to the local machine.

Procedure

Step 1: Obtain the repository URL

Example:

`https://github.com/username/project.git`

Step 2: Clone the repository

`git clone https://github.com/username/project.git`

Step 3: Enter the cloned repository

`cd project`

Step 4: Check the remote repository

`git remote -v`

Result

The remote Git repository was successfully cloned to the local machine.

Program 5: Fetching Changes and Rebasing

Aim: Fetch the latest changes from a remote repository and rebase the local branch onto the updated remote branch.

Procedure

Step 1: Check the current branch

```
git branch
```

Step 2: Fetch the latest remote changes

```
git fetch origin
```

Step 3: Switch to the local branch

```
git switch master
```

Step 4: Rebase onto the updated remote branch

```
git rebase origin/master
```

For repositories using main

```
git rebase origin/main
```

Step 5: View the history

```
git log --oneline --graph --all
```

Notes

- If a conflict occurs after resolving it: `git add <resolved-file>`
- Continue the rebase: `git rebase --continue`
- Cancel the rebase: `git rebase --abort`

Result

The latest remote changes were fetched and the local branch was rebased onto the updated remote branch.

Program 6: Merging with a Custom Commit Message

Aim: Merge "feature-branch" into "master" while providing a custom merge commit message.

Procedure

Step 1: Switch to master

```
git switch master
```

Step 2: Merge the feature branch with a custom message

```
git merge --no-ff feature-branch -m "Merge feature branch into master"
```

Step 3: View the merge history

```
git log --oneline --graph
```

Notes

- --no-ff forces creation of a merge commit.
- -m specifies the custom commit message.

Result

The feature-branch was successfully merged into master with a custom commit message.

Program 7: Creating a Lightweight Git Tag

Aim: Create a lightweight Git tag named "v1.0" for a commit in the local repository.

Procedure

Step 1: View the current commits

```
git log --oneline
```

Step 2: Create the lightweight tag

```
git tag v1.0
```

Step 3: List tags

```
git tag
```

Step 4: View tag information

```
git show v1.0
```

Notes

- To tag a specific commit: `git tag v1.0 <commit-id>`

Result

A lightweight Git tag named v1.0 was successfully created.

Program 8: Cherry-Picking a Range of Commits

Aim: Cherry-pick a range of commits from "source-branch" into the current branch.

Procedure

Step 1: View commits in source-branch

```
git log source-branch --oneline
```

Example commit history

```
a111111 Commit A  
b222222 Commit B  
c333333 Commit C  
d444444 Commit D
```

Step 2: Switch to the destination branch

```
git switch master
```

Step 3: Cherry-pick commits B through D

```
git cherry-pick b222222^..d444444
```

Notes

- The range b222222^..d444444 includes commits B, C, and D.
- If conflicts occur, resolve them, stage the files, and continue with git cherry-pick --continue.
- To cancel an in-progress cherry-pick: git cherry-pick --abort

Result

The selected range of commits from source-branch was successfully applied to the current branch.

Program 9: Viewing Details of a Specific Commit

Aim: Given a commit ID, view the author, date, commit message, and changes introduced by that commit.

Procedure

Step 1: Display complete commit details

```
git show <commit-id>
```

Example

```
git show abc123
```

Step 2: Display only commit information

```
git show --no-patch abc123
```

Step 3: Display selected information in a custom format

```
git show -s --format="%H%n%an%n%ad%n%s" abc123
```

Notes

- %H = commit hash
- %an = author name
- %ad = author date
- %s = commit subject/message

Result

The details of the specified commit were successfully displayed.

Program 10: Displaying Commits by Author and Date

Aim: Display all commits made by the author "JohnDoe" between January 1, 2025 and December 31, 2025.

Procedure

Step 1: Display matching commits

```
git log --author="JohnDoe" --since="2025-01-01" --until="2025-12-31 23:59:59"
```

Step 2: Display the results in compact form

```
git log --author="JohnDoe" --since="2025-01-01" --until="2025-12-31 23:59:59" --oneline
```

Result

All commits matching the specified author and date range were displayed.

Program 11: Displaying the Last Five Commits

Aim: Display the last five commits in the repository history.

Procedure

Step 1: Display the last five commits

```
git log -5
```

Step 2: Display them in compact form

```
git log -5 --oneline
```

Result

The five most recent commits in the repository history were successfully displayed.

Program 12: Reverting a Commit

Aim: Undo the changes introduced by the commit with the ID "abc123".

Procedure

Step 1: Revert the specified commit

```
git revert abc123
```

Step 2: Check repository status

```
git status
```

Step 3: View the updated history

```
git log --oneline
```

Notes

- git revert creates a new commit that reverses the changes introduced by abc123.
- This approach preserves the existing commit history and is commonly used for commits that have already been shared.
- If Git opens an editor for the revert commit message, save the message and close the editor.

Result

The changes introduced by commit abc123 were successfully undone by creating a new revert commit.

Quick Git Command Reference

Operation	Command
Initialize repository	git init
Check status	git status
Add file	git add <filename>
Commit changes	git commit -m "message"
View history	git log
Create branch	git branch feature-branch
Switch branch	git switch feature-branch
Merge branch	git merge feature-branch
Stash changes	git stash
List stashes	git stash list
Apply stash	git stash apply
Clone repository	git clone <repository-url>
Fetch remote changes	git fetch origin
Rebase	git rebase origin/master
Create tag	git tag v1.0
Cherry-pick range	git cherry-pick <range>
Show commit	git show <commit-id>
Last five commits	git log -5
Revert commit	git revert <commit-id>